1 / THE PRODUCTS

OSMO connects tasks into a workflow

Use OSMO when several container tasks need to work together.

It handles their resources, dependencies and data.

Simulate
Generate robot data

→

Train
Build a model

→

Evaluate
Check the model

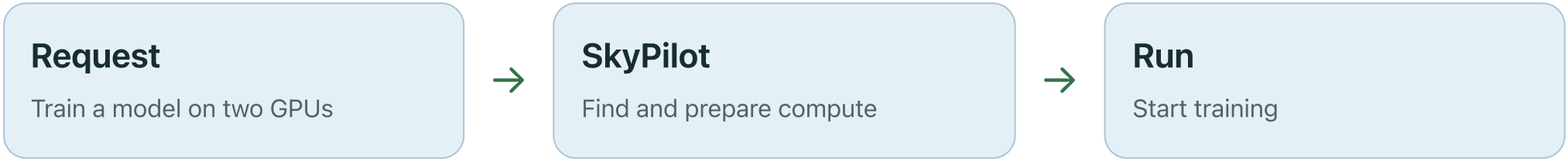
> The output of one task becomes the input of the next.

1 / THE PRODUCTS

SkyPilot finds compute and runs your job

Use SkyPilot to run a task across clouds or Kubernetes.

It chooses suitable compute from the infrastructure you allow and prepares the environment.



> These resources form a SkyPilot compute cluster. It can run another job later.

2 / THE METRICS GAP

We want to see usage for that work

This example covers workloads running as Pods in Kubernetes.

OSMO

Workflow → tasks

How much GPU did this workflow and its tasks use?

SKYPILOT

Compute cluster → roles

How much GPU did this compute cluster and its head / worker roles use?

> To answer, connect each measured Pod to its platform grouping.

1. Recorded metadata association

2 / THE METRICS GAP

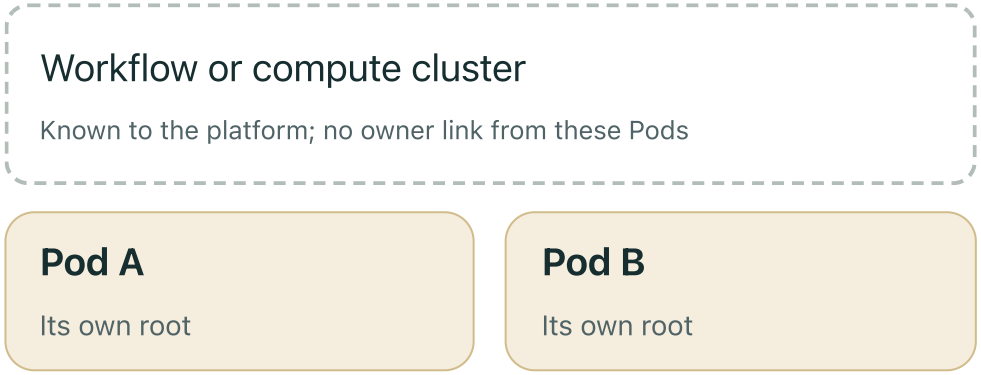
The platform grouping is outside Kubernetes ownership

A KUBERNETES WORKLOAD



Karta follows the owner links to the JobSet.

THE CAPTURED OSMO / SKYPILOT PODS



With the generic Pod definition, Karta reports each Pod separately.

> Karta sees the Pods. It cannot infer their common platform grouping.

1. OSMO captured objects 2. SkyPilot captured objects 3. Karta ownership contract

2 / THE METRICS GAP

Illustrative values

The Pods already carry useful labels

OSMO POD

`osmo.workflow_uuid`

W

`osmo.task_name`

train

Group by workflow.
Then show each task.

SKYPILOT POD

`skypilot-cluster-name`

C

`ray-node-type`

head

Group by compute cluster.
Then show each role.

Short example IDs. Scope each key to its platform installation and Kubernetes cluster / namespace.

1. OSMO labels
2. SkyPilot labels

3 / WAYS TO SOLVE IT

Available approach

Option 1: join the metadata in Prometheus

Enable the needed Pod labels in kube-state-metrics.

Join them to Karta's Pod metrics by namespace and Pod UID.

Karta Pod mapping + Pod labels → **Prometheus** Join the same Pod → **Grouped view** Workflow or compute cluster

Include Kubernetes cluster identity when querying several clusters.

> This metadata join worked in the lab. Each Pod keeps its real UID.

1. Tested metadata join rules
2. Recorded results

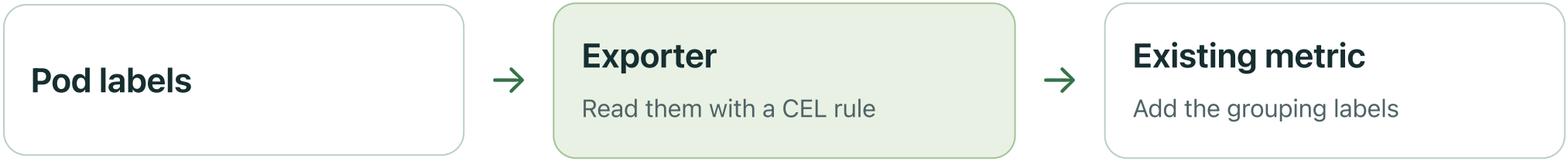
3 / WAYS TO SOLVE IT

Proposed

Option 2: let the exporter read the labels

Issue #518 proposes configurable CEL expressions.

CEL lets a rule read a field from a Pod or workload.



> The exporter would attach the grouping information directly.

Proposed work. It does not replace the join to GPU measurements.

1. Generic metric labels proposal #518

3 / WAYS TO SOLVE IT

Proposed Helm extension

OSMO: match the Pod, then set the metric labels

The exporter would run these rules only for its generic Pod fallback. Each value under labels is CEL.

```
exporter:
  metricLabelRules: # Proposed
    - name: osmo
      metrics: [karta_pod_workload_info]
      sources: [pod]
      matchExpression: |
        has(object.metadata.labels) &&
        ["osmo.workflow_uuid", "osmo.task_name"].all(k,
          k in object.metadata.labels && object.metadata.labels[k] != "")
      labels:
        workflow_id: object.metadata.labels["osmo.workflow_uuid"]
        component: "'task'"
        component_instance: object.metadata.labels["osmo.task_name"]
        replica_group_id: '""'
```

INPUT: POD LABELS

```
osmo.workflow_uuid: W
osmo.task_name: train
```

METRIC LABELS: BEFORE

```
component="pod"
component_instance=""
```

METRIC LABELS: AFTER

```
workflow_id="W"
component="task"
component_instance="train"
replica_group_id=""
```

No match or evaluation error: keep the base labels. Native Karta mappings, workload / Pod UIDs and phase stay unchanged.

This syntax extends #518, which currently rejects built-in overrides. Task names are display values; use the task UUID when grouping logical tasks.

3 / WAYS TO SOLVE IT

Proposed Helm extension

SkyPilot: map head and worker into component

The same rule shape reads SkyPilot's compute-cluster label and Pod role.

```
exporter:
  metricLabelRules: # Proposed
    - name: skypilot
      metrics: [karta_pod_workload_info]
      sources: [pod]
      matchExpression: |
        has(object.metadata.labels) &&
        "skypilot-cluster-name" in object.metadata.labels &&
        object.metadata.labels["skypilot-cluster-name"] != "" &&
        "ray-node-type" in object.metadata.labels &&
        object.metadata.labels["ray-node-type"] in ["head", "worker"]
      labels:
        compute_cluster: object.metadata.labels["skypilot-cluster-name"]
        component: object.metadata.labels["ray-node-type"]
        component_instance: '""'
        replica_group_id: '""'
```

INPUT: POD LABELS

```
skypilot-cluster-name: C
ray-node-type: head
```

METRIC LABELS: BEFORE

```
component="pod"
component_instance=""
```

METRIC LABELS: AFTER

```
compute_cluster="C"
component="head"
component_instance=""
replica_group_id=""
```

A worker Pod produces component="worker". These labels identify compute, not a SkyPilot job.

Re-evaluate on Pod updates. If the match stops or evaluation fails, use the base labels again. Kubernetes objects are not changed.

4 / PHASE AND CLUSTERS

Read workflow and job phase from the platform

OSMO

Workflow and task status

```
osmo workflow query WORKFLOW
```

OSMO tracks dependencies, retries and completion. A waiting task may have no Pod yet.

SKYPILOT

Job status

```
sky queue CLUSTER
sky jobs queue
```

The first reads jobs on a compute cluster. The second reads managed jobs and their tasks.

> A SkyPilot Pod can stay Running after its job finishes. Pod phase is not job phase.

The generic Pod status remains Pod status. To add platform phase, a separate connector would read the platform API. CEL over Pod labels cannot fetch it.

1. OSMO workflow status API 2. OSMO lifecycle 3. SkyPilot job status 4. Managed job status

4 / PHASE AND CLUSTERS

Both manage several clusters. One task has a placement.

OSMO

Workflow → pool → one backend

In the inspected version, every task in a submitted workflow uses that backend's Kubernetes cluster.

Different workflows can use different backends. Cross-pool workflows are tracked in OSMO #879.

SKYPILOT

Task attempt → one compute cluster

On Kubernetes, its head and worker Pods use one Kubernetes cluster.

Separate pipeline tasks or recovery attempts can use other allowed clusters.

> Collect Pod metrics in each Kubernetes cluster, then query them together.

Keep Kubernetes cluster and platform installation identity. A shared name alone is not a safe cross-cluster key.

1. OSMO pool selects backend 2. OSMO applies backend to every task 3. OSMO cross-pool request #879 4. SkyPilot multiple Kubernetes clusters 5. SkyPilot managed pipelines

3 / WAYS TO SOLVE IT

Start with the platform grouping

Now	Use the metadata join to group Pod identities today.
Next	Implement #518 to put grouping labels directly in exporter metrics.
If needed	Add component overrides for common component panels.

OSMO: workflow → task

SkyPilot: compute cluster → role

Each Pod keeps its real Kubernetes identity.

Karta supplies identity; DCGM supplies GPU measurements. Shared Pod usage cannot be divided into jobs from these labels alone. The lab verified metadata mapping, not GPU usage or successful SkyPilot job execution.

1. Metadata association approach
2. Exporter proposal